

დავალება: Project Feedback Platform - Git და React ინტეგრაცია

მიზანი: ისწავლოთ გუნდური მუშაობა GitHub-ის გამოყენებით, სწორი Git ისტორიის (Commits, Branches, Pull Requests) შექმნა და React (Vite) პროექტის ნულიდან აწყობა მზა ბექენდზე.

ნაბიჯი 1: რეპოზიტორიის შექმნა და ბექენდის ატვირთვა (გუნდის ლიდერი)

- გადმოწერეთ ჩემ მიერ გამოგზავნილი დაზიპული ფაილი (project-feedback-backend.zip) და ამოაარქივეთ.
- შექმენით ახალი, ცარიელი რეპოზიტორია GitHub-ზე.
- ლოკალურად, თქვენს ფოლდერში გააკეთეთ git init, დაამატეთ ბექენდის ფაილები და გააკეთეთ პირველი კომიტი. (მაგ: git commit -m "chore: initial commit with backend").
- ატვირთეთ (Push) კოდი GitHub-ზე.
- GitHub-ის სეთინგებიდან დაამატეთ თქვენი მენეჯილე(ები) როგორც Collaborators, რათა მათაც შეძლონ კოდზე წვდომა და ცვლილებების შეტანა.

ნაბიჯი 2: ფრონტენდის ინიციალიზაცია (ახალ ბრენჩზე)

- ყურადღება: main ბრენჩზე პირდაპირ არ ვწერთ! ყველა ახალ ფიჩერზე ვქმნით ახალ ბრენჩს (მაგ: git checkout -b feature/init-frontend).
- შექმენით ახალი React პროექტი Vite-ით და TypeScript-ით (ფოლდერის სახელი: project-feedback-frontend).
 - დააინსტალირეთ Tailwind CSS (v4) და გააკეთეთ მისი საბაზისო კონფიგურაცია.
 - დააკომიტეთ ცვლილებები აღწერილობითი მესიჯით (მაგ: feat: setup Vite React app with Tailwind CSS).
 - ატვირთეთ ბრენჩი, გააკეთეთ Pull Request (PR) და დაამერჯეთ main-ში. მენეჯილემ უნდა განიხილოს (Review) და დაადასტუროს PR.

ნაბიჯი 3: გარემოს ცვლადების (Environments) კონფიგურაცია

- მენეჯილემ უნდა ჩამოტვირთოს განახლებული კოდი (git pull origin main) და შექმნას ახალი ბრენჩი: feature/environments.
- ფრონტენდის მთავარ დირექტორიაში შექმენით .env ფაილი (ლოკალური სერვერისთვის) და განერეთ ბექენდის URL (VITE_API_URL=http://localhost:3000).
- src ფოლდერში შექმენით ფოლდერი environments და შიგნით ფაილი environment.ts, სადაც დააქსპორტებთ ამ URL-ს.
- დააკომიტეთ (feat: add environment configurations) და დაამერჯეთ main-ში PR-ის გავლით.

ნაბიჯი 4: Routing და გვერდების სტრუქტურა

1. ისევ განაახლეთ **main** ბრენჩი ლოკალურად და შექმენით ახალი: **feature/routing**.
2. დააინსტალირეთ **react-router-dom**.
3. შექმენით **src/pages** ფოლდერი და შიგნით მინიმუმ 3 ფაილი: **Landing.tsx**, **Login.tsx** და **Signup.tsx**.
4. შექმენით **src/routes/AppRoutes.tsx** და გაწერეთ ნავიგაციის ლოგიკა. **App.tsx**-ში გამოიძახეთ მხოლოდ **AppRoutes**.
5. გაუკეთეთ გვერდებს **W3Schools**-ის სტილის მარტივი და სუფთა **Tailwind** დიზაინი (მუქი ფონი, მწვანე ღილაკები).
6. დააკომიტეთ ცვლილებები (მაგ: **feat: setup react router and base pages**) და დაამერჯეთ **PR**-ის საშუალებით.

ნაბიჯი 5: Components და Pages დაშორება (Component Architecture)

1. შექმენით ახალი ბრენჩი: **feature/components**.
2. **React**-ში მნიშვნელოვანია კოდის მრავალჯერადად გამოყენება (**Reusability**). შექმენით **src/components** ფოლდერი.
3. ამოიღეთ განმეორებადი **UI** ელემენტები **Pages** ფაილებიდან (მაგალითად: **Input** ველები, მთავარი მწვანე ღილაკი) და შექმენით მათთვის ცალკეული ფაილები (მაგ: **Button.tsx**, **InputField.tsx**).
4. თქვენს **Landing.tsx**, **Login.tsx** და **Signup.tsx** ფაილებში გამოიძახეთ (**Import**) ეს შექმნილი კომპონენტები ნაცვლად იმისა, რომ ერთი და იგივე **HTML/Tailwind** კოდი ბევრჯერ დააკოპიროთ.
5. დააკომიტეთ (**feat: extract reusable UI components**) და დაამერჯეთ **PR**-ის გავლით **main**-ში.